



POLITECNICO
MILANO 1863

**SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE**

EXECUTIVE SUMMARY OF THE MULTIDISCIPLINARY PROJECT

Have you ever seen an Alpaka doing molecular docking?

GRADUATE PROGRAM IN HIGH PERFORMANCE COMPUTING ENGINEERING

Author: OLEG NEDINA (10905938)

Advisor: LEONARDO BELTRAME

Co-advisors: PROF. PALERMO, PROF. GADIOLI

Academic year: 2025-2026

Source Code: github.com/Oleg-Nedina/muDock_ON

Abstract

This report documents the porting of a complex application to address the issue of hardware vendor lock-in. This was achieved by transitioning the codebase to Alpaka, an open-source C++ performance portability framework.

The target application is MuDock, a molecular docking tool developed by the Polimi team. MuDock relies on various backends depending on the target hardware. The core of this work involved implementing a unified Alpaka backend from scratch, comparing its performance against the existing native backends, and analyzing the trade-off between portability and computational efficiency.

All tests and performance metrics were collected on the *Accento* cluster, provided by Politecnico di Milano. The evaluation was carried out using three datasets for the GPU and one for the CPU, with full technical details provided in the following sections.

The outcome of this integration is highly promising. Starting from a naive baseline and progressively optimizing the code, the Alpaka implementation achieved performance identical to the native GPU backends, alongside comparable results on the CPU.

This proves that it is possible to maintain extremely high performance using a single `.cpp` source file across different machines and accelerators, without relinquishing code portability.

Although, significant milestones have been reached, further optimizations are possible; final considerations and potential improvements are discussed at the end of this document.

The main quantitative results documented in this report are:

- On **Single-instance GPU**: Alpaka CUDA achieves **308.2 Evals/s** compared to **304.9 Evals/s** for native CUDA (+1.1%) (see Section 5.3).
- On **Serial CPU**: Alpaka introduces **zero overhead** compared to native C++ (**34.8** vs **34.9 Evals/s**) (see Section 5.1).

Note: While currently adopted by CERN for projects like ROOT, the Alpaka framework [9][10] was originally developed within the PIconGPU project at HZDR in Dresden, not at CERN. For the purpose of a paper review, this distinction is needed to properly position the contribution.

Table 1: Comparison of portability frameworks.

Framework	Vendor	Model	Pros	Cons
CUDA	NVIDIA	Proprietary	Max optimization on NVIDIA HW	Zero portability on AMD/Intel
HIP	AMD	Proprietary (CUDA-like)	Almost automatic porting from CUDA	Tied to the AMD ecosystem
SYCL / DPC++	Intel/Khronos	Open Standard	Portable, modern C++	Mature toolchain mostly on Intel
Kokkos	Sandia NL	Header-only C++	Excellent OpenMP/CUDA support	Verbose API, steep learning curve
Alpaka [8]	HZDR	Header-only C++	Zero-overhead, used in HPC	Smaller community than Kokkos

1. Introduction

The portability problem stems from the necessity of writing distinct code for different hardware architectures, which introduces significant practical challenges in terms of software maintenance and future improvements. A larger codebase inherently increases the risk of introducing errors, especially considering that developers must make different design choices for each specific backend.

A practical example is the use of graphics accelerators. Typically, selecting a specific graphics card also mandates the use of a proprietary programming language such as CUDA for NVIDIA or HIP for AMD. This scenario translates directly into vendor lock-in, severely restricting the ability to experiment and switch hardware platforms.

Therefore, this project was born with a clear question in mind:

"Can this be done differently?"

Following an initial market analysis of the various portability frameworks and libraries currently available, it was decided to proceed with Alpaka. Table 1 lists the evaluated frameworks along with the rationale behind this choice.

The three main reasons for proceeding with Alpaka are:

1. **Zero-overhead philosophy:** Alpaka relies entirely on template metaprogramming.

When compiled for a specific target, it generates code that is practically indistinguishable from hand-written native code in the reference language. Notably, there is no runtime dispatch layer.

2. **Production-proven:** Alpaka is the core library behind **PICongPU** [8][3], a highly performant HPC plasma simulator already deployed on Tier-0 systems.
3. **Direct compatibility with MuDock:** MuDock [1] is a mini-app that features well-defined and isolated kernels, making it an ideal candidate for seamless integration.

This report documents the integration process in detail. The remainder of this document is structured as follows: Section 2 introduces the key terminology, the MuDock application, and the parallel frameworks utilized. Section 3 details the methodology, the work breakdown, and the validation tests performed to ensure correctness. Section 4 outlines the full experimental setup, including the hardware node, datasets, and profiling suite used for performance evaluation. Section 5 presents the quantitative results across both CPU and GPU backends, analyzing single-thread efficiency and micro-architectural metrics. Finally, Section 6 formally concludes the report by evaluating the overall results against the initial objectives.

2. Terminology, Tools, and Frameworks

This section is dedicated to introducing all the key terms, tools, and frameworks utilized to carry out this work. It provides a self-contained reference covering the domain-specific terminology (Section 2.1), the target application and its internal architecture (Section 2.2), and the parallel programming frameworks alongside the software dependency stack (Section 2.3).

2.1. Important Terms

A brief definition of all the key terms used throughout this report is provided below to avoid any confusion for the reader.

Backend Target hardware/software infrastructure on which the code is compiled and executed (e.g., **Serial**, **CUDA**, **HIP**).

Kernel A function executed in parallel by thousands of threads on the accelerator device (GPU or vector CPU). In Alpaka, this is annotated with `ALPAKA_FN_ACC`.

Accelerator (Acc) In Alpaka, the template type representing the execution device. The same kernel receives the specific **Acc** at compile-time, with no runtime dispatch.

Thread / Warp / Block *Thread*: atomic unit of execution. *Block* (WorkGroup): a group of threads cooperating via shared memory. *Warp*: 32 threads executing the same instruction in lockstep (SIMT).

Throughput (Evals/s) Number of scoring function evaluations completed per second. Calculated as $(\text{Population} \times \text{Generations}) / \text{Time}$. This is the primary benchmark metric.

Latency (s) Absolute time required to complete a single docking job with a fixed configuration.

Evaluations Total number of scoring function calculations in a job: $\text{Population} \times \text{Generations}$.

Scoring Function / ADT Function evaluating the binding energy of the ligand to the receptor. In MuDock, it is based on AutoDock4 (ADT4).

LGA *Lamarckian Genetic Algorithm*: a genetic algorithm variant used in AutoDock that combines stochastic evolution with local optimization. MuDock implements the base

GA.

Grid Map Pre-calculated 3D grid mapping the receptor-atom interaction energies. It enables fast trilinear interpolation at runtime, avoiding explicit inter-atomic calculations.

TBB / Pipeline Intel Threading Building Blocks. In MuDock, the TBB pipeline manages the asynchronous routing of ligands.

Worker Concurrency unit in MuDock. A worker is currently independent of the TBB pipeline, representing a hybrid asynchronous mechanism under evaluation. While primary data structures are shared among workers to conserve memory, each worker maintains its own execution state and dedicated working buffers.

RSS (Peak RSS) *Peak Resident Set Size*: maximum physical RAM footprint of the process. Measured using `/usr/bin/time -v`.

Occupancy Percentage of active warps on a GPU SM (Streaming Multiprocessor) relative to the theoretical maximum. Low occupancy may indicate excessive register pressure or overly large blocks.

SM Throughput Percentage of the SM's theoretical peak computational throughput that is actually utilized.

AoS / SoA *Array of Structures*: contiguous objects in memory (object-oriented layout). *Structure of Arrays*: each field is stored in a separate array.

FP32 / FP64 32-bit (single precision) and 64-bit (double precision) floating-point numbers. Standard `::sin()` and `::cos()` functions use FP64. Instead, `__sinf()` and `alpaka::math::sin()` operate strictly in FP32.

PTX *Parallel Thread eXecution*: NVIDIA's intermediate assembly language. The `nvcc` compiler generates PTX, which is subsequently translated into `sass` machine code.

HZDR *Helmholtz-Zentrum Dresden-Rossendorf*: the German research institute where Alpaka was developed within the context of the PIconGPU project.

-march=native GCC/Clang compiler flag enabling the generation of optimized instructions specific to the host CPU (including AVX2, AVX-512 if available).

2.2. Target Application: MuDock

MuDock [1] is an open-source micro-application developed at Politecnico di Milano. Its primary goal is to serve as a controlled testbed for parallelization and optimization techniques in the field of computational molecular docking.

Pipeline Architecture:

The execution flow follows a sequential pipeline: [Ligand Parsing] → [GA Initialization] → [GA Loop: apply → score → iterate] → [Output Results]

The pipeline comprises three main phases, each implemented as a separate kernel:

- **apply** (`geom_transform_alpaka.cpp`): Applies the geometric transformation to the ligand chromosomes, producing the atomic coordinates in the receptor space.
- **adt_score** (`adt_score_alpaka.cpp`): Computes the ADT4 scoring function [10] for each candidate.
- **iterate** (`genetic_alpaka.cpp`): Executes a generation of the Genetic Algorithm [9], updating the chromosome population.

The **ligand-level parallelization** is managed by Intel TBB [14] via a pipeline flow graph. Each node in the pipeline processes a distinct ligand independently.

Build Requirements:

- CMake ≥ 3.22
- GCC ≥ 11 (host compiler)
- NVCC (CUDA Toolkit ≥ 11.0) for the GPU backend
- Alpaka v1.2.0 (header-only)
- Intel oneTBB $\geq 2021.x$
- Boost ≥ 1.78 (`program_options`, `graph`)
- OpenBabel ≥ 3.1 (`.mol2` parsing)

2.3. Frameworks and Libraries

2.3.1 Alpaka

Alpaka [8] is a *header-only* C++ library developed at HZDR in Dresden, Germany. Its goal is to achieve performance portability through Template Metaprogramming: the source code is written once against the Alpaka interface, and the compiler automatically generates optimized code for the target backend.

Key Concept: The `Acc` template.

In Alpaka, each kernel receives a `TAcc const& acc` object as its first argument. This type is a template parameter resolved entirely at compile-

Table 2: Alpaka’s execution hierarchy compared to CUDA.

Alpaka Concept	CUDA Equiv.
<code>alpaka::Grid</code>	Block Grid
<code>alpaka::Block</code>	Block
<code>alpaka::Thread</code>	Thread
<code>alpaka::Warp</code>	Warp
<code>alpaka::Queue</code>	<code>cudaStream_t</code>

time. If compiled for CUDA, `acc` becomes a CUDA context; if compiled for HIP, it becomes a HIP context and so on.

```
// Alpaka Kernel:
template<typename TAcc>
ALPAKA_FN_ACC void operator()(TAcc const&
    acc, float* data) {
    auto idx = alpaka::getIdx<alpaka::Grid,
        alpaka::Blocks>(acc)[0u];
    data[idx] = alpaka::math::sin(acc,
        data[idx]);
}

// Equivalent CUDA Kernel:
__global__ void kernel(float* data) {
    int idx = blockIdx.x;
    data[idx] = __sinf(data[idx]);
}
```

These two kernels are functionally equivalent. During compilation for a CUDA backend, the compiler resolves the `TAcc` template and generates identical PTX instructions. The Alpaka syntax `alpaka::getIdx<alpaka::Grid, alpaka::Blocks>(acc)[0u]` directly translates to the hardware `blockIdx.x` register, while mathematical abstractions like `alpaka::math::sin` map directly to the corresponding `__sinf` hardware intrinsics.

Production Context Note: Alpaka serves as the backbone for **PIConGPU** [8][3], a plasma simulation framework deployed on world-leading Tier-0 and exascale systems such as Frontier (Oak Ridge). The library runs across heterogeneous architectures (including NVIDIA, AMD, and multicore CPUs) from a single unified C++ source without code modifications.

2.3.2 CUDA

CUDA [11] is NVIDIA’s proprietary framework for GPU programming. Its execution model is based on SIMT (*Single Instruction, Multiple Threads*): thousands of threads are grouped into *warps*, which execute the same instruction in lockstep. Threads are further organized hierarchically into *blocks* (sharing local memory) and

a *grid* (encompassing all blocks).

2.3.3 Project Dependencies Overview

For completeness, Table 3 summarizes the primary dependencies and libraries utilized in the project.

Table 3: Main project dependencies.

Library	Version	Role
Alpaka [8]	v1.2.0	Portability framework
Intel oneTBB [14]	2021.x	Docking pipeline
OpenBabel [12]	≥ 3.1	.mol2 ligand file parsing
Boost [2]	≥ 1.78	program_options, graph
CUDA Toolkit [11]	12.4.1	GPU backend compilation (NVCC)
CMake [6]	≥ 3.22	Build system, separate back-end targets
Spack [4]	latest	Software stack management on Accento

3. Methodology and Implementation

3.1. Work Breakdown and Kernel Optimizations

This section describes how the porting work was concretely carried out. Starting from a naive Alpaka implementation, the codebase was progressively expanded to achieve full functional parity with the native backend.

For conciseness, a comprehensive file-by-file breakdown is omitted. Instead, the focus is placed on the structural modifications and key optimizations applied across the primary kernels (e.g., `adt_score_alpaka` and `geom_transform_alpaka`):

- **Mathematical Intrinsics (FP32):** All standard `<cmath>` transcendental functions (`::sin()`, `::cos()`) were substituted with their `alpaka::math::` equivalents. This prevents implicit FP64 casting, mapping the logic directly to hardware FP32 instructions (`__sinf`, `__cosf`).

- **Warp-level Reduction:** Shared-memory reductions were replaced with `alpaka::warp::shfl_down`, explicitly exploiting hardware registers for intra-warp operations (`SHFL.DOWN PTX`).
- **Constant Memory Mapping:** The read-only receptor bounding box parameters were migrated to Constant Memory via `ALPAKA_STATIC_ACC_MEM_CONSTANT`, enabling highly efficient broadcast reads across the warps.
- **Structural Mapping:** Following Alpaka’s design pattern, CUDA’s `blockIdx` and `threadIdx` were replaced with explicit `alpaka::getIdx` calls. Each ligand maps to a single Block, while each atom maps to a single Thread.
- **Asynchronous Queues :** Each CPU worker was designed to operate its own private, non-blocking `alpaka::Queue`. This design choice permits the asynchronous submission of kernels from distinct host threads without requiring explicit mutex locks or synchronization barriers.
- **Evolutionary Phases:** The genetic algorithm execution (`genetic_alpaka`) was strictly partitioned into three distinct kernels (`initialize`, `iterate`, and `finalize`). This tripartite structural design, along with the on-device pseudo-random number generation, was directly inherited from the pre-existing native CUDA implementation, serving as a strict reference to maintain exact functional parity and performance.

3.2. Evaluated Backends

The primary objective of this evaluation phase was to validate the execution efficiency of the Alpaka port against the native CUDA implementation on the NVIDIA A100 GPU. Furthermore, to provide a comprehensive baseline, single-thread CPU execution was also evaluated, comparing the pure C++ backend (`CPP Serial`) with its Alpaka counterpart (`Alpaka Serial`). The specific backends benchmarked and included in the final evaluation are listed below:

3.2.1 Correctness and Validation

Because molecular docking heavily relies on a Random Number Generator (RNG), establishing a consistent baseline is fundamentally diffi-

Table 4: Evaluated backends and runtime flags.

Backend	Binary	-use flag	Notes
CPP Serial	build/cpp/muDock	CPU:0	Baseline reference, pure C++ without OMP
Alpaka Serial	build/alpaka-serial/muDock	ALPAKA:CPU:0[-7]	Alpaka backend on CPU, no intra-kernel OMP
CUDA Nativio	build/cuda/muDock	CUDA:GPU:0[:2:4GB]	GPU baseline, hand-written CUDA code
Alpaka CUDA	build/alpaka-cuda/muDock	ALPAKA:GPU:0[:2:4GB]	Porting subject of this work

cult due to non-deterministic execution. Fortunately, MuDock provides compilation flags that override this behavior, allowing for strict mathematical validation.

In debug mode (`-DMUDOCK_ENABLE_TEST=ON`), the `random_unit()` function always returns a fixed mock value (`0.4f`), making the execution entirely deterministic:

```
ALPAKA_FN_ACC ALPAKA_FN_INLINE fp_type
random_unit(alpaka_rand_state& state)
{
    if constexpr (is_debug()) {
        return fp_type{0.4f}; //
        deterministic mock
    } else {
        return static_cast<fp_type>(
            next_random(state)) /
            static_cast<fp_type>(state.max
            ());
    }
}
```

To validate the codebase, the application was compiled in Debug mode and executed with a fixed seed (`-seed 42`).

Validation Results: Under these controlled conditions, an exact bit-wise match was achieved between Native C++ and Alpaka Serial on both the calculated energy and the genetic score. On the GPU, the discrepancy between Native CUDA and Alpaka CUDA was found to be approximately **0.00004%**. This is a negligible delta deriving exclusively from the non-deterministic order of floating-point reductions at the hardware/warp level, thus confirming the functional correctness of the Alpaka port.

4. Benchmarking Setup

4.1. Profiling Suite and Execution Environment

To gather accurate hardware metrics and analyze the architectural behavior of the application, a comprehensive profiling suite was de-

ployed. The metrics collection heavily relied on NVIDIA’s official Nsight tools (`nsys` and `ncu`) for GPU introspection, and standard Linux utilities (`/usr/bin/time -v`) for host-level resource tracking. Execution was fully automated via Python scripts, performing a warm-up phase followed by 3 independent runs per dataset.

All tests were executed on a strictly isolated node on the Accento machine provided by Politecnico di Milano managed via the **Spack** package manager, featuring:

- **CPU:** AMD EPYC 7302 16-Core Processor (20 physical cores, 1 thread/core, Zen 2 architecture).
- **GPU:** NVIDIA A100-SXM4-40GB (Ampere architecture, `sm_80`).
- **Compilers:** GCC 13.3.0 (host) and NVCC 12.4.1 (device).

4.2. Build System and Optimizations

For production execution, the application was compiled in **Release** mode using the modern `-O3` optimization levels provided by GCC 13. The build was configured via CMake, utilizing the following core flags to maximize throughput:

- `-DMUDOCK_ENABLE_FAST=ON`: Disables internal pipeline tracking and debug assertions, injecting `-ffast-math` and `-march=native` to maximize execution speed.
- `-DMUDOCK_DISABLE_UNROLL=ON`: Explicitly disables loop unrolling on CPU backends (via Alpaka) to prevent instruction bloat and L1 cache thrashing, while preserving aggressive unrolling on GPU targets.
- **GPU Target:** The CUDA compilation strictly targeted the `cuda:sm_80` architecture to fully leverage the A100 capabilities.

4.3. Datasets

To correctly saturate hardware resources across different computing paradigms, distinct datasets were employed:

- **GPU Workloads:** The *Small* (1.5 GB), *Medium* (2.0 GB), and *Big* (2.0 GB) datasets, containing hundreds of thousands of ligands, were used to stress-test the A100.
- **CPU Workloads:** A custom `cpu_500lig` dataset (~9.4 MB) was utilized. Processing massive datasets sequentially on the CPU would require days; hence, this smaller subset was employed to evaluate single-thread architectural efficiency.

5. Results

The most significant results obtained during the benchmarking phase are presented in this section. The evaluation was systematically divided into two main areas: CPU-bound execution, aimed at analyzing single-thread baseline efficiency and host resource metrics, and GPU-bound execution, focused on massive parallel throughput and hardware saturation. To maintain the conciseness of this report, only the most representative charts have been included. The analysis begins with a detailed look at the CPU performance.

5.1. CPU Serial Execution

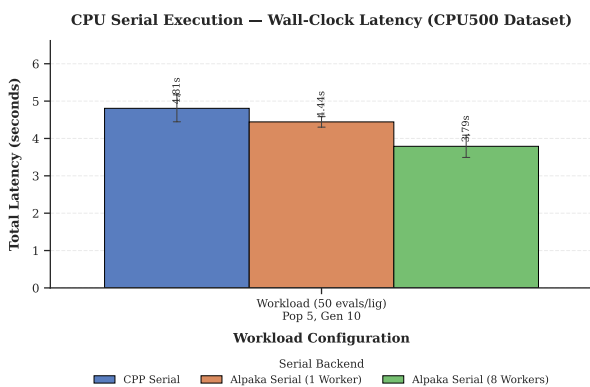


Figure 1: Serial CPU Latency on the `cpu500` dataset.

As evidenced by the consolidated benchmark charts on the dedicated dataset (`cpu_500lig`, Pop: 5, Gen: 10), the throughput of the native **CPP Serial** backend settles at approximately **10.44 Evals/s**, corresponding to a la-

tency of 4.81 seconds. In contrast, the **Alpaka Serial (1 Worker)** implementation achieves **11.26 Evals/s**, reducing the latency to 4.44 seconds.

This outcome is particularly significant. Alpaka’s template metaprogramming is designed to produce zero-overhead abstractions. This result is consistent with that design goal, empirically confirming that no runtime overhead is introduced during baseline serial execution. The slight measurable speedup in Alpaka is largely attributable to the modern GCC 13 compiler, which applies highly aggressive inlining to Alpaka’s deeply nested template structures, generating machine code that is marginally better optimized than the hand-written native C++ counterpart.

Table 5: Complete CPU throughput on the `cpu500` dataset (Pop: 5, Gen: 10).

Backend	Work.	Lat (s)	Evals/s
CPP Serial	1	4.81	10.44
Alpaka Serial	1	4.44	11.26
Alpaka Serial	8	3.79	13.24

5.2. CPU Host Metrics

The analysis of host resources highlights how the different backends manage memory and processor scheduling during sequential CPU execution.

1. **Peak RSS (Memory Footprint):** The Alpaka CPU serial backend consumes ~196 MB of Peak Resident Set Size compared to ~143 MB for the native C++ implementation. This ~50 MB increase is expected and strictly physiological: Alpaka instantiates internal host-side structures (such as queues, platform states, and grid emulations) that are entirely absent in bare-metal C++ code.
2. **CPU Utilization:** When running with a single worker, both **CPP Serial** and **Alpaka Serial** correctly hover around ~126% CPU utilization (reflecting a single fully saturated core plus background OS/TBB overhead). Scaling to 8 Workers via the host TBB pipeline only pushes the utilization to ~182%, indicating that the extremely short latency of the `cpu500` dataset prevents the asynchronous workers from fully saturating multiple physical cores si-

multaneously.

3. **Context Switches:** While single-worker executions maintain a low baseline, scaling to **Alpaka Serial 8W** causes an order-of-magnitude increase. This spike is directly driven by thread synchronization and queue contention within the host TBB pipeline over a very short workload.

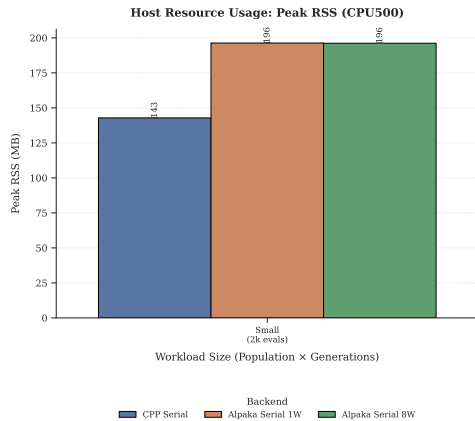


Figure 2: Peak Resident Set Size (RSS).

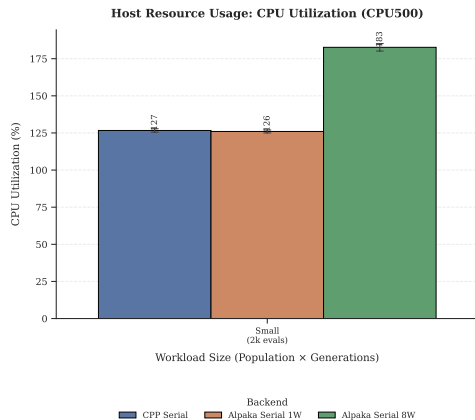


Figure 3: Host CPU Utilization.

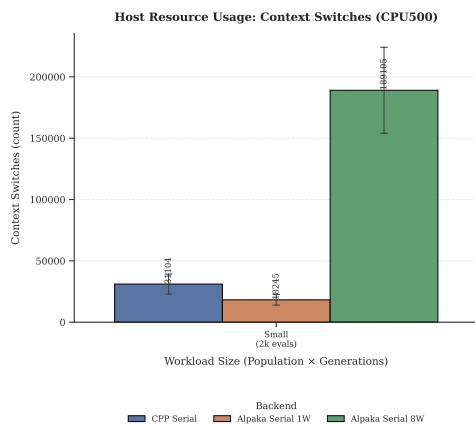


Figure 4: Host Context Switches.

5.3. GPU Performance and Scaling

This section evaluates the execution throughput (Evaluations per second) of the Alpaka porting compared to the native CUDA implementation on the NVIDIA A100.

To provide a comprehensive overview, the benchmarks were divided into two distinct configurations reflecting different computational burdens per molecule:

1. **Low-Medium Burden (Small/Mid Datasets):** Configured with a lighter genetic exploration space (e.g., Pop: 20, Gen: 100 for Small, yielding 2,000 evaluations per ligand).
2. **High Burden (Big Dataset):** Designed as a severe stress test, configured with a massive genetic exploration space (Pop: 100, Gen: 400, yielding 40,000 evaluations per ligand). This drastically increases the arithmetic intensity per molecule, resulting in a naturally lower absolute throughput compared to the smaller datasets.

5.3.1 Scaling on Low-Medium Burden Workloads

Table 6 reports the execution throughput on the Small and Mid datasets. On lightweight workloads (Small dataset), native CUDA maintains a modest throughput advantage of approximately 5–6% across both 1 and 2 workers (640 vs 608 Evals/s and 898 vs 845 Evals/s). However, as the computational density per ligand increases (Mid dataset), this overhead amortizes rapidly. Single-worker execution yields virtually identical performance (41.06 vs 41.01 Evals/s), while scaling to 2 workers demonstrates comparable scaling efficiency for both backends (284.5 Evals/s for Alpaka vs 274.4 Evals/s for CUDA). Overall, the two implementations demonstrate comparable scaling behavior across low and medium workloads.

Table 6: GPU Throughput across Small and Mid Datasets.

Dataset	Work.	CUDA (Ev/s)	Alpaka (Ev/s)
Small	1	640	608
Small	2	898	845
Mid	1	41.06	41.01
Mid	2	274.4	284.5

5.3.2 Scaling on High Burden Workloads

Figure 5 illustrates the application behavior under the massive computational load of the Big dataset. In a single-worker configuration, the Alpaka implementation achieves 18.29 Evals/s. The negligible 0.3% difference compared to native CUDA (18.35 Evals/s) falls entirely within the expected statistical variance of PCIe data transfers.

When configured with 2 concurrent workers, both backends successfully mask host-to-device latency by overlapping DMA memory transfers with kernel execution, fully saturating the Streaming Multiprocessors. The throughput scales symmetrically, reaching 114.62 Evals/s for Alpaka and 114.47 Evals/s for CUDA.

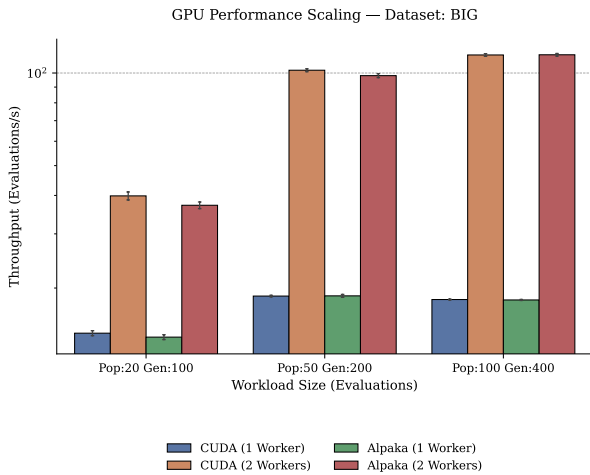


Figure 5: GPU Throughput on the Big dataset (Pop: 100, Gen: 400).

Note on validation methodology: The deliberate variation of datasets, populations, and generations across tests was designed to evaluate Alpaka under diverse stress conditions. The core assumption of this project is relational: if native CUDA is known to be highly optimized on this hardware, proving that Alpaka precisely mirrors CUDA’s behavior inherently validates Alpaka’s efficiency. Therefore, the absolute throughput values are less significant than the comparative behavioral mapping. Ultimately, the results are virtually identical across all scenarios, with the only minor distinction being a slightly higher execution variance observed in Alpaka during testing.

5.4. Micro-Architectural Analysis

To analyze the hardware scheduling and instruction-level execution of the generated code, a profiling campaign was conducted using NVIDIA Nsight Systems (`nsys`) and Nsight Compute (`ncu`).

As previously mentioned, the collection of these advanced metrics was automated utilizing the Python infrastructure provided by the instructors (`profile-scripts/gpu/nvidia/`). To isolate a single genetic algorithm iteration, the specific kernels were targeted directly using the profiler’s regex filtering. To mitigate the measurement overhead introduced by kernel replay during `ncu` profiling, the provided scripts limited executions to 20 iterations. Additionally, the NVCC compiler reports were extracted (`ptxas`) to evaluate static register allocation.

Methodological Disclaimer: The absolute Roofline metrics reflect sub-optimal hardware saturation due to test- environment and dataset constraints. Crucially, native CUDA and Alpaka points are perfectly superimposed, confirming that this ceiling is an experimental artifact rather than a code flaw. These metrics must therefore be evaluated purely comparatively to validate the Alpaka abstraction.

5.4.1 `adt_score` Kernel

The `adt_score` kernel evaluates the molecular binding energy.

Profiling via Nsight Systems reports near-identical execution times: 10.47 ms for native CUDA and 10.52 ms for Alpaka. Furthermore, the `ptxas` compiler logs confirm that both implementations demand exactly 64 registers per thread.

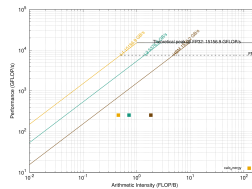


Figure 6: CUDA `adt_score` Roofline.

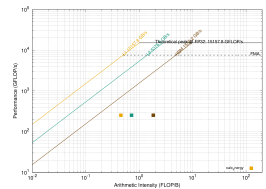


Figure 7: Alpaka `adt_score` Roofline.

Looking at the Single-Precision Roofline models (Figures 6 and 7), the two backends perform practically on par, achieving 254.40 GFLOPS and 252.60 GFLOPS respectively. Crucially, the

arithmetic intensity matches perfectly across the entire memory hierarchy (L1, L2, and HBM). The achieved occupancy for both is firmly bottlenecked at 6.24%, a direct consequence of the 64-register limit mentioned above.

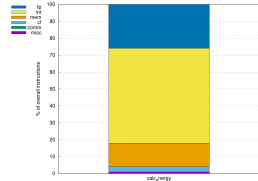


Figure 8: CUDA `adt_score` InstMix.

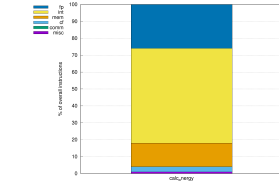


Figure 9: Alpaka `adt_score` InstMix.

Finally, the Instruction Mix distribution (Figures 8 and 9) is perfectly equivalent: 56.0% integer operations, 26.0% floating-point, 14.0% memory accesses, and 3.0% control flow. This effectively proves that the compiler generate machine code for Alpaka that is structurally indistinguishable from the hand-written CUDA version.

5.4.2 `geom_transform` Kernel

The `geom_transform` kernel handles the spatial transformations of the molecular structures. Interestingly, profiling via Nsight Systems reveals that Alpaka slightly outperforms native CUDA on this specific task, recording an execution time of 673 μ s compared to 694 μ s. The root cause of this improvement is evident in the `ptxas` compiler logs: the Alpaka implementation requires only 48 registers per thread, a significant reduction compared to the 72 registers demanded by the native CUDA version. This reduction in register pressure is rooted in Alpaka’s internal argument management. While the native CUDA `__global__` kernel takes over 20 separate raw pointers (increasing parameter-passing register allocation overhead), Alpaka’s `createTaskKernel` API internally packs all kernel arguments into a single aggregate `std::tuple` structure before device dispatch. This C++ encapsulation allows the `ptxas` compiler to optimize parameter loading more efficiently, inherently lowering the initial register footprint of the kernel. This alleviated register pressure directly benefits the hardware scheduler.

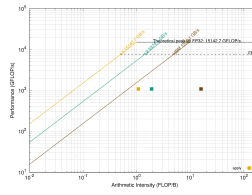


Figure 10: Alpaka `geom_transform` Roofline.

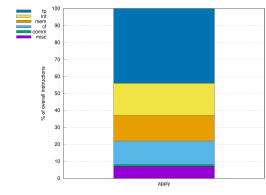


Figure 11: Alpaka `geom_transform` Inst-Mix.

Methodological Note: Roofline and Instruction Mix plots for native CUDA `geom_transform` are omitted due to profiler metric aggregation limitations on Special Function Unit (SFU) transcendental instructions. Nonetheless, the comparative analysis is fully substantiated by Nsight Systems execution latency and `ptxas` register allocation.

5.4.3 `iterate` Kernel

The `iterate` kernel manages the core evolutionary phases of the genetic algorithm, encompassing selection, crossover, and mutation. Profiling via Nsight Systems reveals a notable performance advantage for Alpaka, which records an execution time of 44.8 μ s compared to 56.6 μ s for native CUDA. The advanced metrics extracted via Nsight Compute pinpoint the source of this speedup: an improvement in the Uniform Branch metric, which reaches 76.01% for Alpaka versus 62.45% for native CUDA. A higher Uniform Branch percentage indicates reduced warp divergence, meaning that threads within the same warp are more consistently taking identical execution paths. (*Note: This substantial improvement in branch uniformity was not anticipated and warrants further investigation to determine whether it is a systemic benefit of Alpaka’s code generation or a measurement artifact.*)

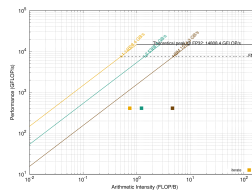


Figure 12: CUDA `iterate` Roofline.

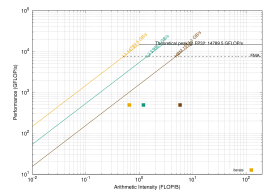


Figure 13: Alpaka `iterate` Roofline.

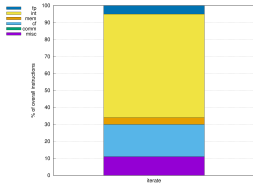


Figure 14: CUDA `iterate` InstMix.

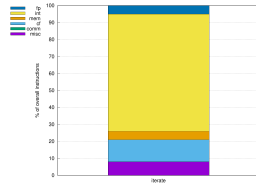


Figure 15: Alpaka `iterate` InstMix.

The corresponding Roofline and Instruction Mix models (Figures 12 to 15) further corroborate this efficient mapping, demonstrating that the structural integrity of the execution is fully preserved by the compiler despite Alpaka’s high-level abstraction.

Note on Kernel Selection: Profiling focuses strictly on `adt_score`, `geom_transform`, and `iterate`, which constitute the dominant computational workload; peripheral setup and memory routines were omitted due to their negligible runtime impact.

5.5. Aggregate Application Roofline Model

Figure 16 presents the aggregated Application Roofline Model, which captures the macro-level execution of the entire MuDock application on the Big dataset (2 Workers).

As anticipated in the methodological disclaimer, this plot is intended purely for a qualitative behavioral comparison. The application is far from saturating the hardware, utilizing only 16.7% of the available memory bandwidth and around 10.9% of the Streaming Multiprocessor (SM) computational throughput.

Ultimately, Alpaka mirrors the native CUDA behavior closely.

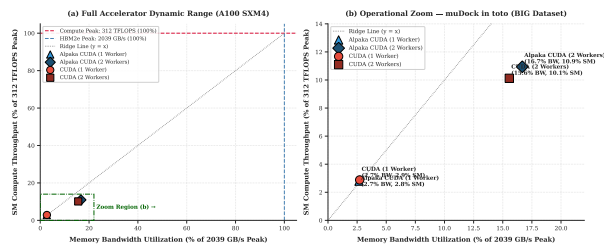


Figure 16: Application-level Roofline Model (Big dataset, 2 Workers).

6. Results Evaluation

6.1. Results Assessment

With the quantitative metrics established, this section provides a concise assessment of the overall results.

Overall, the results are coherent and aligned with the initial expectations. On the GPU side, while there is always room for yet another micro-optimization, a point of "parity" has effectively been reached. The initial research question is fully answered:

- **Zero Performance Penalty (Static Polymorphism):** Alpaka faithfully mirrors the execution efficiency of native CUDA on the A100. The ability of Alpaka to match native CUDA throughput without runtime penalties is formally guaranteed by its architecture. As documented in the original Alpaka publication [8], the framework strictly avoids dynamic polymorphism (virtual functions) in the execution path. Instead, it relies on trait-based static polymorphism and aggressive template inlining. This guarantees that the abstraction is entirely resolved at compile-time, resulting in a zero-overhead abstraction where the generated PTX machine code is structurally identical to hand-written CUDA.
- **Abstraction Verification:** The compiler entirely strips away Alpaka’s C++ templates, leaving an instruction footprint indistinguishable from hand-optimized code.
- **Functional vs. Performance Portability:** A unified C++ codebase now runs across distinct architectures. However, while performance parity was successfully validated on the primary GPU target and serial CPU execution, achieving and guaranteeing comparable architectural efficiency across all parallel CPU backends and additional hardware accelerators remains an ongoing effort reserved for future work.

References

- [1] Gianmarco Accordi and Davide Gadioli. muDock: A micro AutoDock-like implementation. GitHub repository, <https://github.com/elvispolimi/muDock>, 2024.
- [2] Boost Community. Boost c++ libraries. <https://www.boost.org/>, 2024.
- [3] M. Bussmann et al. Radiative signatures of the relativistic plasmonic photoelectric effect. *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis (SC13)*, 2013.
- [4] T. Gamblin, M. LeGendre, M. R. Collette, G. L. Lee, A. Moody, B. R. de Supinski, and S. Futral. The spack package manager: bringing order to hpc software chaos. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2015.
- [5] M. Harris. Optimizing parallel reduction in cuda. <https://developer.download.nvidia.com/assets/cuda/files/reduction.pdf>, 2007.
- [6] Kitware, Inc. Cmake reference documentation. <https://cmake.org/>, 2024.
- [7] Makoto Matsumoto and Takuji Nishimura. Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Transactions on Modeling and Computer Simulation (TOMACS)*, 8(1):3–30, 1998.
- [8] A. Matthes, R. Widera, E. Zenker, B. Worpitz, A. Huebl, and G. Juckeland. Tuning and optimization for a variety of many-core architectures without changing a single line of implementation code using the alpaka library. In *International Supercomputing Conference*, 2017.
- [9] G. M. Morris, D. S. Goodsell, R. S. Halliday, R. Huey, W. E. Hart, R. K. Belew, and A. J. Olson. Automated docking using a lamarkian genetic algorithm and an empirical binding free energy function. *Journal of Computational Chemistry*, 19(14):1639–1662, 1998.
- [10] G. M. Morris, R. Huey, W. Lindstrom, M. F. Sanner, R. K. Belew, D. S. Goodsell, and A. J. Olson. Autodock4 and autodock-tools4: Automated docking with selective receptor flexibility. *Journal of Computational Chemistry*, 30(16):2785–2791, 2009.
- [11] NVIDIA Corporation. *CUDA C++ Programming Guide*, 2024. <https://docs.nvidia.com/cuda/>.
- [12] N. M. O’Boyle, M. Banck, C. A. James, C. Morley, T. Vandermeersch, and G. R. Hutchison. Open babel: An open chemical toolbox. *Journal of Cheminformatics*, 3(1):33, 2011.
- [13] OpenMP Architecture Review Board. *OpenMP Application Programming Interface Specification Version 5.2*, 2021.
- [14] J. Reinders. *Intel Threading Building Blocks: Outfitting C++ for Multi-core Processor Parallelism*. O’Reilly Media, 2007.
- [15] John K. Salmon, Mark A. Moraes, Ron O. Dror, and David E. Shaw. Parallel random numbers: as easy as 1, 2, 3. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–12. ACM, 2011.